

Reasoning-Based Traversal of an Unknown Graph: Wikipedia Game AI

Caleb Bailey, James Irwin, Nathan Lim, Kyra Ramirez, Colby Watts
California Polytechnic State University, College of Engineering
CSC 581: Computer Support for Knowledge Management
Dr. Alex Bisberg
March 20th, 2026

Abstract

This project investigates whether large language model (LLM)-based agents can navigate Wikipedia using only the hyperlinks available on the current page. Rather than giving the system access to a precomputed graph or a shortest-path solver during navigation, we treat Wikipedia traversal as a partially observable search problem. At each step, the agent observes the current page, reasons about promising links, and chooses the next page to visit. We implemented and evaluated three agent designs: a Default agent, a Planning agent, and a Tree-of-Thoughts (ToT) agent. To assess path quality, we compared the agents against human performance and against shortest paths computed separately with a BFS-based graph-search method. The results show that LLM-guided navigation is feasible under local information constraints and that structured search significantly improves performance. Overall, the project highlights both the potential and the limitations of using LLMs as reasoning-based search agents in graph-like environments.

Introduction & Overview

The Wikipedia Game is a well-known internet game in which a player moves from one Wikipedia article to another using only the hyperlinks visible on each page. While the rules are simple, the task becomes challenging when the start and target pages are only loosely related. That makes the game a useful setting for studying reasoning, planning, and search. Our project explores whether an LLM-based system can solve this task under the same constraints as a human player. The system does not have access to Wikipedia's full hyperlink graph during navigation. Instead, it must make decisions one step at a time using only the current page and its outgoing links. This creates a constrained search problem in which the agent must balance local reasoning with long-term path quality.

More broadly, this project is relevant to AI because many real systems operate under incomplete information. They must act using local observations, imperfect heuristics, and limited memory of the broader environment. In that sense, Wikipedia navigation is more than a game; it is a compact testbed for sequential reasoning and agent design. This project is also closely tied to course themes. It combines LLM reasoning, tool use, iterative decision-making, and search-based agent behavior. It also naturally extends into more advanced topics such as structured exploration, graph-of-thought reasoning, and multi-agent systems.

Background & Related Work

Wikipedia is a strong fit for this project because the MediaWiki Action API provides reliable access to article content and outgoing hyperlinks. This allows the system to retrieve useful page-level information while limiting the agent to the links available on the current page. That setup aligns well with our task: the agent must navigate step by step using local information rather than a global map.

From a classical AI perspective, this project relates to breadth-first search, depth-first search, heuristic search, and graph traversal. In our system, however, BFS is used only for evaluation. The agents do not use BFS during navigation. Instead, they select links using local page context and LLM-based reasoning. After each run, we compare the resulting path against a shortest path computed separately with an existing graph-based method. This provides a clear way to measure how efficient the agent’s decisions were relative to the optimal solution.

Recent LLM research also informed the project. Search-on-Graph frames graph reasoning as an observe-then-navigate process, which aligns closely with our design. Prior work evaluating LLMs on Wikipedia navigation similarly suggests that language models can compete with humans on constrained navigation tasks. These studies helped motivate the project, but our work focuses more directly on implementation and comparison across multiple agent designs. Graph-based orchestration frameworks such as LangGraph also influenced our thinking, especially for branching search and structured exploration. Although we did not fully migrate the system to LangGraph during the quarter, its workflow model helped shape how we approached more advanced agent behavior.

Features, Requirements, & Evaluation Criteria

At a high level, the system is designed to accept a Wikipedia start page and target page, then navigate between them using only the links available on each visited page. It must produce a valid path, avoid unnecessary loops, and expose enough information for the user to understand what the agent considered and why it made each decision. To make that goal testable, we defined a set of concrete requirements. The system must operate in an observe-reason-act loop, use only outgoing links from the current page, log the pages it visits and the candidate links it considers, and stop either when the target is reached or when a maximum step limit is exceeded. It must also support multiple agent variants so that different strategies can be compared on the same tasks.

Evaluation focuses on measurable outcomes. We track whether the agent reaches the target, how many hops it takes, how long the run lasts, and how close the final path is to the shortest known path. Our evaluation criteria are summarized in *Table 1* on the following page.

Table 1. Traceability from Features to Requirements and Evaluation

Feature	Requirement	Evaluation Metric
Navigate between Wikipedia pages using only visible links	Agent may only use outgoing hyperlinks from the current page	Success/failure, valid path check
Use LLM reasoning to choose the next step	Agent must operate in an observe-reason-act loop	Path quality, reasoning logs
Avoid repeated dead ends and cycles	System must track and prevent repeated-page loops	Failure analysis, path validity
Support multiple navigation strategies	Multiple agent configurations must be runnable and comparable	Cross-agent comparison
Expose what the system is doing	System must log visited pages, candidate links, and rationale	Readability and reproducibility
Reach the target efficiently	System must stop on success or max-step failure	Hops, runtime, comparison to shortest path

System Design & Architecture

The system is organized around an iterative navigation loop. A user provides a start page and a target page. The system retrieves information from the current page, selects candidate links, and asks an LLM-based agent to choose the next step. The controller updates the run state and repeats the process until the target is reached or the run terminates. The flowchart of our system architecture is captured in **Figure 1**, and the major system components of our model are broken down and explained in *Table 2* on the following page.

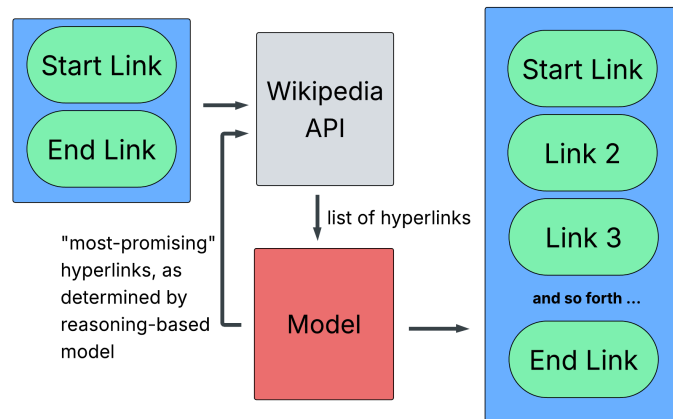


Figure 1. Overall System Architecture for LLM-Based Wikipedia Navigation

Table 2. Major System Components

Component	Purpose	Main Responsibilities
User Input Layer	Accepts run configuration	Receives start page, target page, and optional settings
Game Controller	Manages the navigation loop	Validates pages, tracks visited pages, updates path history, checks stop conditions
Wikipedia Client	Interfaces with MediaWiki API	Retrieves page text, outgoing links, and normalized titles
LLM Reasoning Module	Selects the next page	Evaluates candidate links and returns the most promising choice
Multi-Path Search Logic	Supports structured exploration	Maintains and scores candidate branches for advanced agents
Evaluation Harness	Measures performance	Records hops, runtime, success rate, and baseline comparisons
Run-State Data Model	Stores run information	Tracks visited pages, step history, rationale, and outcome

Implementation & Prototype

The final prototype was implemented primarily in Python. We used the MediaWiki API for Wikipedia access, Tinker for LLM inference, and HTML for the front-end interface. By the end of the project, the system could accept a start page and target page, run a selected agent, and output the resulting navigation path and logs as shown in **Figure 2**. Importantly, (and not shown in Figure 2), the UI also includes a console log at the end of each run that shows the exact reasoning trace for our agent setups.

Wikipedia Game AI

(Developed by The Code Ninjas)

Start Wikipedia page title

Target Wikipedia page title

Minecraft

Chess

Agent

LLM candidates

Model

default

28

Qwen3-30B - 30B Instruction (default)

Run

Stop

Result

Success in 3 hops (14.55s) Target: Chess

Each hop prints the exact **anchor text** you can Ctrl+F on the current Wikipedia page, then click.

Current path:

Minecraft --["Genres"]--> Video game genre --["turn based"]--> Turn-based game --["chess"]--> Chess

Steps (links clicked):

Start: Minecraft
Step 1: Genres
Step 2: turn based
Step 3: chess
Destination: Chess

Path Statistics

Optimal shortest path (via Wikipath) 2 hops

Minecraft --> Scrabble --> Chess

Agent Final Path Performance Metrics

Metric	Value
Optimal path length	2
LLM path length	3
Efficiency (optimal / LLM)	66.7%
Number of shortest paths that exist	5
Time elapsed	14.55s

Figure 2. Front-End Interface for the Wikipedia Game AI Prototype

The three implemented agents were designed to represent increasing levels of reasoning and search structure, allowing the project to compare how added planning and branch exploration affected navigation performance. While each agent operated within the same overall system and used the same Wikipedia interface, they differed in how candidate links were generated,

evaluated, and selected at each step. *Table 3* summarizes the core idea behind each agent, along with its primary strengths and main limitations.

Table 3: Comparison of Implemented Agent Designs

Agent	Core Idea	Strength	Main Limitation
Default Agent	Filters page links and lets the LLM choose the next page	Simple and fast baseline	Can be misled by weak local candidates
Planning Agent	Generates hypotheses and retrieves stronger candidate links before selection	Better candidate quality and more deliberate reasoning	Higher runtime due to more overhead
Tree-of-Thoughts (ToT) Agent	Explores a tree of promising pages using structured search	Best path quality and better recovery from weak choices	More complex control logic

The Default Agent served as the baseline system. On each page, it scrapes all visible outgoing links, computes a simple heuristic score using basic word matching for each and selects the top- k candidates. It sorts the links based on that heuristic to ensure that the target page or any very close matches to it will always be shown at the top of the list. It then passes only these high-scoring links to the LLM, which chooses the next hop and explains its reasoning. This design is lightweight (no embeddings, no external retrieval index), fast to run, and established that purely local, LLM-guided navigation could reliably reach the target from many starting points. The only hyperparameter available for the Default Agent is the number of candidate links to provide to the LLM.

The Planning Agent was designed to address a specific issue of our default agent—with smaller parameter models, we faced issues where the model would become “overwhelmed” when we fed it a large number of links (exceeding a few hundred). To account for this, we wanted to design an agent specifically designed to benefit smaller parameter models by reducing the number of links we provide to the LLM, but while still ensuring that those fewer links still consisted of the most promising paths to the target page. Our final design for the Planning Agent adds an explicit retrieval and planning layer before the final decision step. It first parses the current page into section headings and link structure, then asks the LLM to generate hypotheses about which sections/topics are most promising for reaching the target, along with targeted search queries. Those queries are executed over an advanced hybrid retrieval system that fuses semantic and lexical similarity using ranked reciprocal fusion before reranking using a pretrained cross-encoder model. We used free models from HuggingFace to create vector embeddings for

semantic search and for our reranking step, along with the BM25 algorithm for our lexical search. This is a slightly modified TF-IDF algorithm that is very commonly used in retrieval systems. The final top-k most relevant links are then aggregated and deduplicated before presenting a smaller, higher-quality pool to the LLM. This generally improves link relevance and reduces dead-ends at the cost of additional model calls and retrieval latency. We also automatically enforce a search for the target page in addition to the agent’s own search queries, so that anything very closely related to the target page is automatically presented to the LLM and at the top of the list. The only hyperparameter available for the Planning Agent is the number of candidate links provided as a result of each search query (the “top-k” for each individual retrieval step).

The Tree-of-Thoughts Agent is the most structured and search-heavy design. Instead of committing greedily to a single link at each step, it treats pages as nodes in a search tree and maintains a frontier (priority queue) of candidate paths, each scored for promise toward the target. At every step it: (1) uses the default agent’s heuristic to propose many outgoing links from frontier nodes, (2) asks the LLM to score or pick the top-k most promising children, and (3) pushes those children back into the frontier. The agent periodically rescores all frontier nodes with an LLM “global view” prompt and then commits moves along the best-scoring path, backtracking when a different branch looks better. This best-first, multi-path exploration allows the system to very easily recover from weak local choices, systematically explore alternative routes, and, in our experiments, achieved the strongest path efficiency and success on harder paths. Our system allows users to select both the candidates generated at each node as well as the number of expansions per step as hyperparameters.

The final prototype supported user input, Wikipedia retrieval, agent-based selection, path logging, and comparative evaluation against human and shortest-path baselines. Several stretch goals remained unfinished, including richer visualizations, stronger backtracking support, and reinforcement-learning-style optimization. Even so, the project clearly moved beyond a proof of concept and produced a working experimental system.

Validation & Evaluation

We evaluated the system on eight randomly selected Wikipedia navigation tasks. For each start-target pair, we compared human performance, multiple agent configurations, and a shortest-path baseline computed separately using third-party graph-based code. We ran each agent configuration at least 3 times to make sure our results were validated—our final evaluation results consist of both averages and best performance results.

That shortest-path computation was used only after the run as a benchmark. During navigation, the agents still had to act using only local page information. This separation was important

because it allowed us to compare our agents against an optimal solution without giving them access to the full graph during the task itself.

Most of our system failures were due to the LLM failing to follow the instructions within our prompt related to JSON formatting of its link choices/search queries, etc. This was especially apparent with the smaller parameter Qwen3-30B model, which was rarely able to successfully complete a full ToT run due to failure in instruction following. The larger parameter models that we tested with, namely Llama-3.3-70B and GPT-OSS-120b, had far less issues with instruction following.

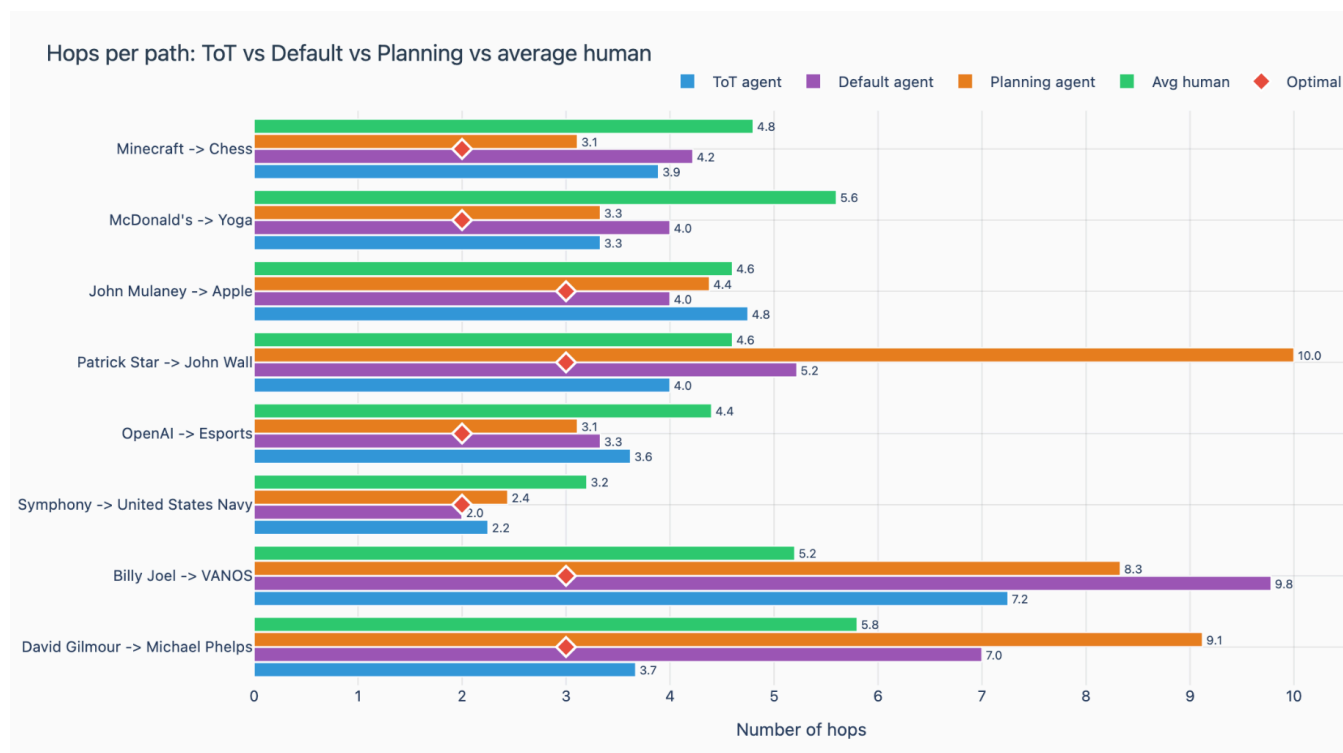


Figure 3. Average Navigation Hop Count between all System Agents and Human Players

Table 4. Average Performance Across Agent and Human Results

Agent	Avg. Hops	Avg. Time (s)	Efficiency (Avg. Hops / Avg. Optimal)	Paths Ever Optimal
Human	4.78	159.4	0.59	2/8
Default	4.94	58.4	0.61	2/8
Planning	4.58	178.0	0.65	3/8

ToT	3.94	44.1	0.67	4/8
-----	------	------	------	-----

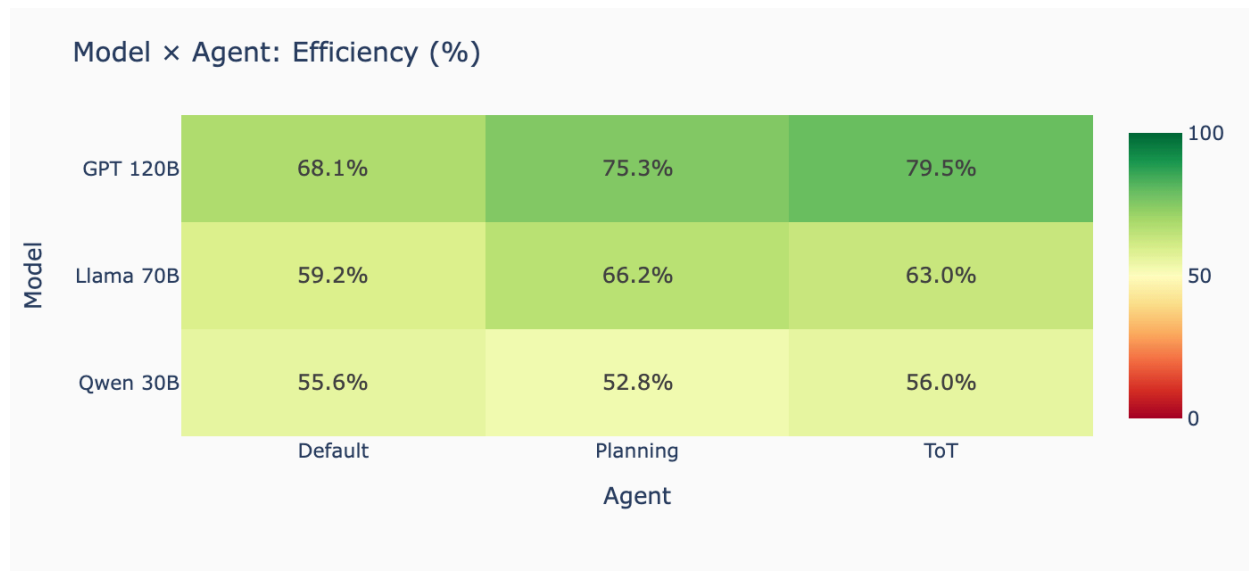


Figure 4. Heatmap of efficiency (avg. hops/avg. optimal) for each model × agent combination.

Our full evaluation results contain too much data to include in this report, but a summary of the most interesting findings are above. For full results, including an in-depth statistics comparison between system and human performance, please see the files in the [evaluation/results directory of our project GitHub repo](#).

Our results indicate that our agent design was always limited by the base LLM that powered the agent. GPT-OSS-120B significantly outperformed the other two LLMs we tested on all agent configurations, which we expect is due to the fact that it is both the highest parameter model out of the three and also the only one that is a reasoning model (generates extensive reasoning tokens before final output, which includes reasoning before providing output to our own chain-of-thought prompts). Our overall results indicate that our ToT agent, across all LLM setups, performed best overall—especially for some of the more complicated paths that took a higher average number of steps across all setups. It achieved the lowest average hop count, the fastest average runtime, and the highest number of optimal matches. The Default agent was fast but somewhat less efficient in path quality, while the Planning agent improved path quality at the cost of longer runtime.

Several conclusions follow from these results. First, LLM-based agents can successfully navigate Wikipedia under local information constraints. Second, structured search improves path quality. Third, there is a clear tradeoff between more deliberate candidate generation and computational

cost. Fourth, different choices for LLMs will lead to clear performance differences even in the exact same environments and agentic pipelines.

Our evaluation also exposed several weaknesses. One major challenge was the size of the branching factor on many Wikipedia pages. Some pages contain hundreds or thousands of outgoing links, which makes it impractical to pass every option directly to the model. That forced the system to rely heavily on filtering and reranking. Another issue was topic drift, where an agent selected a link that looked reasonable locally but was not strategically helpful. The simpler agents were especially vulnerable once they committed to a weak branch, while the ToT agent handled this better by maintaining multiple candidate paths.

Conclusions & Future Work

This project produced a working Wikipedia navigation system that uses LLM-based reasoning to move through an unknown graph one page at a time. The system successfully accepted user-specified start and target pages, retrieved local page information, selected links through multiple agent strategies, and produced measurable results against both human and shortest-path baselines.

One of the clearest lessons from the project is that model reasoning alone is not enough. Candidate selection, search structure, and recovery mechanisms matter just as much. A capable model can still perform poorly if it is given a weak candidate list or forced into a purely greedy process. The strongest version of our system was the one that combined LLM reasoning with an explicit search structure. A second lesson is that Wikipedia navigation is a useful benchmark for agentic AI. It requires sequential reasoning, constrained visibility, tool use, and planning across semantically distant concepts. That makes it much more revealing than a one-shot prompt-response task.

The project also has limitations. The evaluation set was still relatively small, some recovery and visualization features were incomplete, and the system remained sensitive to prompt design and candidate selection quality. For that reason, the final product is best described as a strong prototype rather than a finished research system.

Future work should focus on the improvements most likely to matter. The clearest next steps are systematic prompt-engineering experiments, hyperparameter tuning across agent settings, better visualization of the agent's reasoning process, direct visualization of the path on Wikipedia pages, and a live graph view of explored pages and candidate links. These directions would improve both performance and interpretability.

Overall, the project shows that LLMs can function as effective reasoning-guided search agents in unknown graph environments, especially when paired with structured search and better candidate management.

Project GitHub Repository

Our GitHub repository contains all of our source code for this project, along with full evaluation results. Our UI can be run and tested following the instructions present in the repository README: <https://github.com/nathanlim1/wikipedia-game-ai>

References

- [1] MediaWiki, “API:Action API,” MediaWiki Documentation. [Online]. Available: https://www.mediawiki.org/wiki/API:Action_API. [Accessed: Mar. 15, 2026].
- [2] A. Besta et al., “Search-on-Graph: Iterative Informed Navigation for Large Language Model Reasoning on Knowledge Graphs,” arXiv preprint arXiv:2510.08825, 2025. [Online]. Available: <https://arxiv.org/abs/2510.08825>.
- [3] M. Bragagnolo, A. Fumagalli, and R. Navigli, “Evaluating Large Language Models on Wikipedia Graph Navigation: Insights from the WikiGame,” in Proceedings of the 11th Italian Conference on Computational Linguistics (CLiC-it), 2025. [Online]. Available: <https://aclanthology.org/2025.clicit-1.63.pdf>.
- [4] LangChain, “LangGraph Overview,” LangGraph Documentation. [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/overview>. [Accessed: Mar. 15, 2026].

AI Use Disclosure

ChatGPT was used for formatting, editing, and rephrasing. The system design, implementation, experiments, and reported results were produced by the project team.